

# Commands

What is a command? The simplest answer is that a *command* is a software program that, when executed on the CLI, performs an action on the computer.

To execute a command, the first step is to type the name of the command. Click in the terminal on the right. Type `ls` and hit **Enter**. The result should resemble the example below:

```
sysadmin@localhost:~$ ls
Desktop  Documents  Downloads  Music  Pictures  Public  Templates  Videos
```

**Note:** By itself, the `ls` command lists files and directories contained in the current working directory. At this point, you shouldn't worry too much about the output of the command, instead, focus on understanding how to format and execute commands.

The `ls` command will be covered in complete detail later in the course.

Many commands can be used by themselves with no further input. Some commands require additional input to run correctly. This additional input comes in two forms: *options* and *arguments*.

The typical format for a command is as follows:

```
command [options] [arguments]
```

Options are used to modify the core behavior of a command while arguments are used to provide additional information (such as a filename or a username). Each option and argument is normally separated by a space, although options can often be combined.

Keep in mind that Linux is case-sensitive. Commands, options, arguments, variables, and file names must be entered exactly as shown.

## Arguments

```
command [options] [arguments]
```

An argument can be used to specify something for the command to act upon. If the `ls` command is given the name of a directory as an argument, it lists the contents of that directory. In the following example, the `/etc/ppp` directory is used as an argument; the resulting output is a list of files contained in that directory:

```
sysadmin@localhost:~$ ls /etc/ppp
ip-down.d  ip-up.d
```

The `ls` command also accepts multiple arguments. To list the contents of both the `/etc/ppp` and `/etc/ssh` directories, pass them both as arguments:

```
sysadmin@localhost:~$ ls /etc/ppp /etc/ssh
/etc/ppp:
ip-down.d  ip-up.d
```

```
/etc/ssh:
moduli          ssh_host_dsa_key.pub      ssh_host_rsa_key      sshd_conf
igssh_config
ssh_host_ecdsa_key  ssh_host_rsa_key.pub
ssh_host_dsa_key    ssh_host_ecdsa_key.pub    ssh_import_id
```

## Options

```
command [options] [arguments]
```

Options can be used with commands to expand or modify the way a command behaves. For example, using the `-l` option of the `ls` command results in a *long listing*, providing additional information about the files that are listed, such as the permissions, the size of the file and other information:

```
sysadmin@localhost:~$ ls -l
total 0
drwxr-xr-x 1 sysadmin sysadmin 0 Jan 29 20:13 Desktop
drwxr-xr-x 1 sysadmin sysadmin 0 Jan 29 20:13 Documents
drwxr-xr-x 1 sysadmin sysadmin 0 Jan 29 20:13 Downloads
drwxr-xr-x 1 sysadmin sysadmin 0 Jan 29 20:13 Music
drwxr-xr-x 1 sysadmin sysadmin 0 Jan 29 20:13 Pictures
drwxr-xr-x 1 sysadmin sysadmin 0 Jan 29 20:13 Public
drwxr-xr-x 1 sysadmin sysadmin 0 Jan 29 20:13 Templates
drwxr-xr-x 1 sysadmin sysadmin 0 Jan 29 20:13 Videos
```

Often the character is chosen to be mnemonic for its purpose, like choosing the letter *l* for *long* or *r* for *reverse*. By default, the `ls` command prints the results in alphabetical order, and so by adding the `-r` option, it prints the results in reverse alphabetical order.

```
sysadmin@localhost:~$ ls -r
Videos  Templates  Public  Pictures  Music  Downloads  Documents  Desktop
```

In most cases, options can be used in conjunction with other options. They can be given as separate options, as in `-l -r`, or combined, as in `-lr`. The combination of these two options would result in a long listing output in reverse alphabetical order:

```
sysadmin@localhost:~$ ls -lr
total 32
drwxr-xr-x 2 sysadmin sysadmin 4096 Oct 31 20:13 Videos
```

```
drwxr-xr-x 2 sysadmin sysadmin 4096 Oct 31 20:13 Templates
drwxr-xr-x 2 sysadmin sysadmin 4096 Oct 31 20:13 Public
drwxr-xr-x 2 sysadmin sysadmin 4096 Oct 31 20:13 Pictures
drwxr-xr-x 2 sysadmin sysadmin 4096 Oct 31 20:13 Music
drwxr-xr-x 2 sysadmin sysadmin 4096 Oct 31 20:13 Downloads
drwxr-xr-x 4 sysadmin sysadmin 4096 Oct 31 20:13 Documents
drwxr-xr-x 2 sysadmin sysadmin 4096 Oct 31 20:13 Desktop
```

The order of the combined options isn't important. The output of all of these examples would be the same:

```
ls -l -r
ls -rl
ls -lr
```

By default the `-l` option of the `ls` command displays files sizes in bytes:

```
sysadmin@localhost:~$ ls -l /usr/bin/perl
-rwxr-xr-x 2 root root 10376 Feb  4 2018 /usr/bin/perl
```

If the `-h` option is added the file sizes will be displayed in *human-readable* format:

```
sysadmin@localhost:~$ ls -lh /usr/bin/perl
-rwxr-xr-x 2 root root 11K Feb  4 2018 /usr/bin/perl
```

Options are often single letters; however, sometimes they are words or phrases as well. Typically, older commands use single letters while newer commands use complete words for options. Single-letter options are preceded by a single dash `-` character, like the `-h` option. Full-word options are preceded by two dash `--` characters. The `-h` option also has an equivalent full-word form; the `--human-readable` option.

```
sysadmin@localhost:~$ ls -l --human-readable /usr/bin/perl
-rwxr-xr-x 2 root root 11K Feb  4 2018 /usr/bin/perl
```

## History

When a command is executed in the terminal, it is stored in a history list. This is designed to make it easy to execute the same command, later eliminating the need to retype the entire command.

Pressing the **Up Arrow** `↑` key displays the previous command on the prompt line. The entire history of commands run in the current session can be displayed by pressing **Up** repeatedly to move back through the history of commands that have been run. Pressing the Enter key runs the displayed command again.

When the desired command is located, the **Left Arrow** ← and **Right Arrow** → keys can position the cursor for editing. Other useful keys for editing include the **Home**, **End**, **Backspace** and **Delete** keys.

To view the history list of a terminal, use the `history` command:

```
sysadmin@localhost:~$ date
Wed Dec 12 04:28:12 UTC 2018
sysadmin@localhost:~$ ls
Desktop  Documents  Downloads  Music  Pictures  Public  Templates  Videos
sysadmin@localhost:~$ cal 5 2030
      May 2030
Su Mo Tu We Th Fr Sa
                1  2  3  4
 5  6  7  8  9 10 11
12 13 14 15 16 17 18
19 20 21 22 23 24 25
26 27 28 29 30 31
sysadmin@localhost:~$ history
 1  date
 2  ls
 3  cal 5 2030
 4  history
```

If the desired command is in the list that the `history` command generates, it can be executed by typing an exclamation point `!` character and then the number next to the command, for example, to execute the `cal` command again:

```
sysadmin@localhost:~$ history
 1  date
 2  ls
 3  cal 5 2030
 4  history
sysadmin@localhost:~$ !3
cal 5 2030
      May 2030
Su Mo Tu We Th Fr Sa
                1  2  3  4
```

```
5 6 7 8 9 10 11
12 13 14 15 16 17 18
19 20 21 22 23 24 25
```

If the `history` command is passed a number as an argument, it outputs that number of previous commands from the history list. For example, to show the last three commands:

```
sysadmin@localhost:~$ history 3
6 date
7 ls /home
8 history 3
```

To execute the *n*th command from the bottom of the history list, type `!-n` and hit Enter. For example, to execute the third command from the bottom of the history list execute the following:

```
sysadmin@localhost:~$ !-3
date
Wed Dec 12 04:31:55 UTC 2018
```

To execute the most recent command type `!!` and hit **Enter**:

```
sysadmin@localhost:~$ date
Wed Dec 12 04:32:36 UTC 2018
sysadmin@localhost:~$ !!
date
Wed Dec 12 04:32:38 UTC 2018
```

To execute the most recent iteration of a specific command, type `!n` followed by the name of the command and hit **Enter**. For example, to execute the most recent `ls` command:

```
sysadmin@localhost:~$ !ls
ls /home
sysadmin
```

## Variables

A variable is a feature that allows the user or the shell to store data. This data can be used to provide critical system information or to change the behavior of how the Bash shell (or other commands) work. Variables are given names and stored temporarily in memory. There are two types of variables used in the Bash shell: *local* and *environment*.

# Local Variables

*Local or shell variables* exist only in the current shell, and cannot affect other commands or applications. When the user closes a terminal window or shell, all of the variables are lost. They are often associated with user-based tasks and are lowercase by convention.

To set the value of a variable, use the following assignment expression. If the variable already exists, the value of the variable is modified. If the variable name does not already exist, the shell creates a new local variable and sets the value:

```
variable=value
```

The following example creates a local variable named `variable1` and assigns it a value of `Something`:

```
sysadmin@localhost:~$ variable1='Something'
```

The `echo` command is used to display output in the terminal. To display the value of the variable, use a dollar sign `$` character followed by the variable name as an argument to the `echo` command:

```
sysadmin@localhost:~$ echo $variable1  
Something
```

## 5.4.2 Environment Variables

*Environment variables*, also called *global variables*, are available system-wide, in all shells used by Bash when interpreting commands and performing tasks. The system automatically recreates environment variables when a new shell is opened. Examples include the `PATH`, `HOME`, and `HISTSIZE` variables. The `HISTSIZE` variable defines how many previous commands to store in the history list. The command in the example below displays the value of the `HISTSIZE` variable:

```
sysadmin@localhost:~$ echo $HISTSIZE  
1000
```

To modify the value of an existing variable, use the assignment expression:

```
sysadmin@localhost:~$ HISTSIZE=500  
sysadmin@localhost:~$ echo $HISTSIZE  
500
```

Many variables are available for the Bash shell, as well as variables that affect different Linux commands. A discussion of all variables is beyond the scope of this chapter; however, more shell variables will be covered as this course progresses.

When run without arguments, the `env` command outputs a list of the environment variables. Because the output of the `env` command can be quite long, the following examples use a text search to filter that output.

In a previous example `variable1` was created as a local variable, so the following search in the environment variables results in no output:

```
sysadmin@localhost:~$ env | grep variable1
```

The pipe `|` character passes the output of the `env` command to the `grep` command, which searches the output.

*This text filtering technique will be covered in detail later in the course.*

The `export` command is used to turn a local variable into an environment variable.

```
export variable
```

After exporting `variable1`, it is now an environment variable. It is now found in the search through the environment variables:

```
sysadmin@localhost:~$ export variable1
sysadmin@localhost:~$ env | grep variable1
variable1=Something
```

The `export` command can also be used to make a variable an environment variable upon its creation by using the assignment expression as the argument:

```
sysadmin@localhost:~$ export variable2='Else'
sysadmin@localhost:~$ env | grep variable2
variable2=Else
```

To change the value of an environment variable, use the assignment expression:

```
sysadmin@localhost:~$ variable1=$variable1 ' '$variable2
sysadmin@localhost:~$ echo $variable1
Something Else
```

Exported variables can be removed using the `unset` command:

```
sysadmin@localhost:~$ unset variable2
```

## 5.4.3 Path Variable

One of the most important Bash shell variables to understand is the `PATH` variable. It contains a list that defines which directories the shell looks in to find commands. If a valid command is entered and the shell returns a "command not found" error, it is because the Bash shell was unable to locate a command by that name in any of the directories included in the path. The following command displays the path of the current shell:

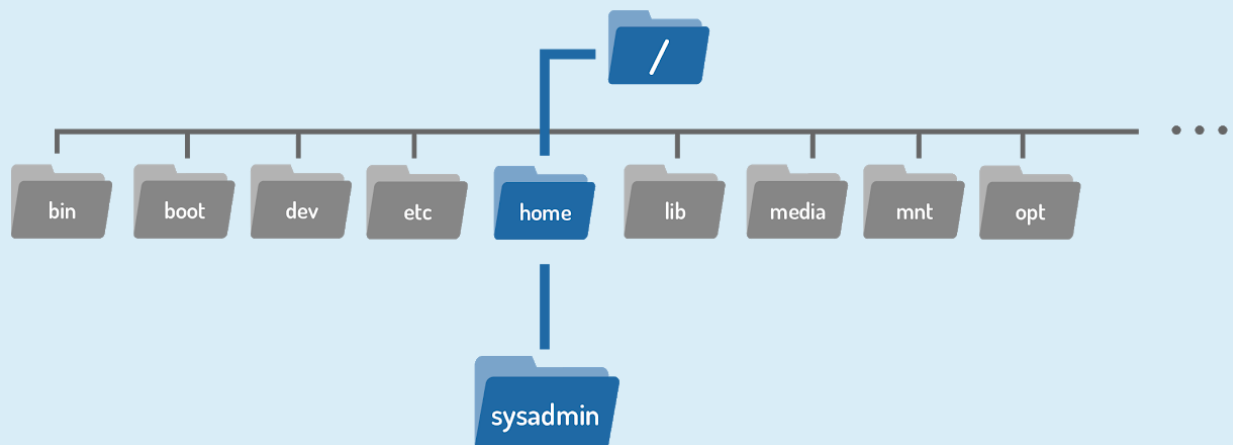
```
sysadmin@localhost:~$ echo $PATH
```

```
/home/sysadmin/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/b
in:/usr/games
sysadmin@localhost:~$
```

Each directory in the list is separated by a colon `:` character. Based on the preceding output, the path contains the following directories. The shell will check the directories in the order they are listed:

```
/home/sysadmin/bin
/usr/local/sbin
/usr/local/bin
/usr/sbin
/usr/bin
/sbin
/bin
/usr/games
```

Each of these directories is represented by a *path*. A path is a list of directories separated by the `/` character. If you think of the filesystem as a map, paths are the directory addresses, which include step-by-step navigation directions; they can be used to indicate the location of any file within the filesystem. For example, `/home/sysadmin` is a path to the home directory:



*Directories and paths will be covered in detail later in the course.*

If the command is not found in any directory listed in the `PATH` variable, then the shell returns an error:

```
sysadmin@localhost:~$ zed
-bash: zed: command not found
sysadmin@localhost:~$
```

If custom software is installed on the system it may be necessary to modify the `PATH` to make it easier to execute these commands. For example, the following will add and verify the `/usr/bin/custom` directory to the `PATH` variable:

```
sysadmin@localhost:~$ PATH=/usr/bin/custom:$PATH
sysadmin@localhost:~$ echo $PATH
/usr/bin/custom:/home/sysadmin/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games
```

When updating the `PATH` variable, always include the current path, so as not to lose access to commands located in those directories. This can be accomplished by appending `$PATH` to the value in the assignment expression. Recall that a variable name preceded by a dollar sign represents the value of the variable.

## Command Types

One way to learn more about a command is to look at where it comes from. The `type` command can be used to determine information about command type.

```
type command
```

There are several different sources of commands within the shell of your CLI including *internal commands*, *external commands*, *aliases*, and *functions*.

## Internal Commands

Also called *built-in commands*, *internal commands* are built into the shell itself. A good example is the `cd` (change directory) command as it is part of the Bash shell. When a user types the `cd` command, the Bash shell is already executing and knows how to interpret it, requiring no additional programs to be started.

The `type` command identifies the `cd` command as an internal command:

```
sysadmin@localhost:~$ type cd
cd is a shell builtin
```

## External Commands

*External commands* are stored in files that are searched by the shell. If a user types the `ls` command, then the shell searches through the directories that are listed in the `PATH` variable to try to find a file named `ls` that it can execute.

If a command does not behave as expected or if a command is not accessible that should be, it can be beneficial to know where the shell is finding the command or which version it is using. It would be tedious to have to manually look in each directory that is listed in the `PATH` variable. Instead, use the `which` command to display the full path to the command in question:

```
which command
```

The `which` command searches for the location of a command by searching the `PATH` variable.

```
sysadmin@localhost:~$ which ls
/bin/ls
sysadmin@localhost:~$ which cal
/usr/bin/cal
```

External commands can also be executed by typing the complete path to the command. For example, to execute the `ls` command:

```
sysadmin@localhost:~$ /bin/ls
Desktop  Documents  Downloads  Music  Pictures  Public  Templates  Videos
```

For external commands, the `type` command displays the location of the command:

```
sysadmin@localhost:~$ type cal
cal is /usr/bin/cal
```

In some cases the output of the `type` command may differ significantly from the output of the `which` command:

```
sysadmin@localhost:~$ type echo
echo is a shell builtin
sysadmin@localhost:~$ which echo
/bin/echo
```

Using the `-a` option of the `type` command displays all locations that contain the command named:

```
sysadmin@localhost:~$ type -a echo
echo is a shell builtin
echo is /bin/echo
```

## Functions

Functions can also be built using existing commands to either create new commands, or to override commands built-in to the shell or commands stored in files. Aliases and functions are normally loaded from the initialization files when the shell first starts.

## Quoting

Quotation marks are used throughout Linux administration and most computer programming languages to let the system know that the information contained within the quotation marks should either be ignored or treated in a way that is very different than it would normally be treated. There are three types of quotes that have special significance to the Bash shell: double quotes `"`, single